

# How to Design a High Performance GEMM Kernel (on B200)?

June 17, 2026

# Talk Outline

- The big picture: GEMM dataflow across an SM.
- The core idea: a GEMM kernel is an *orchestration* problem — containers, signals, and warp roles.
- Key pieces: 2-CTA MMA, epilogue design (TMEM layout + swizzle), and SMEM partitioning.
- From diagram to code, then performance vs cuBLAS (autotuned).
- Adding epilogue fusions: a fused-SwiGLU case study.

# High-Level Design

A GEMM kernel has two major phases: **compute** and **epilogue**.

## Compute

- Fetch A/B tiles.
- Stage tiles in shared memory.
- Issue MMA instructions.
- Accumulate results in TMEM.

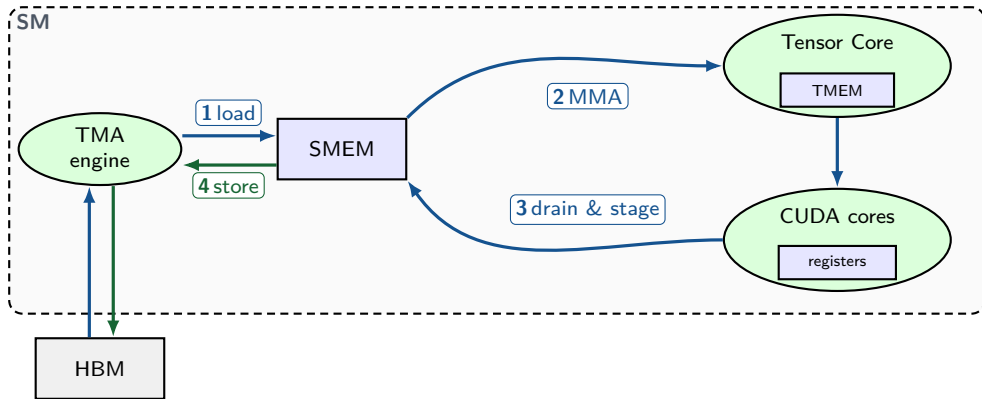
## Epilogue

- Load TMEM into registers.
- Apply optional operations such as ReLU or SwiGLU.
- Pack registers into shared memory.
- Store the final result to HBM.

# The Big Picture: GEMM Dataflow on the SM

Where data lives as it flows through one SM:

**HBM** → **SMEM** → **TMEM** → **registers** → **SMEM** → **HBM**.



Engines own their storage — **TMEM** in the tensor core, **registers** in the CUDA cores; **SMEM** is the shared hub.

# The Core Idea: Orchestration

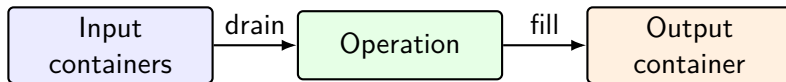
If there is one idea to take away from this talk, it is this:

**A high-performance GEMM kernel is an orchestration problem.**

The kernel is a dataflow schedule over hardware units, warp roles, and on-chip containers. Performance depends on keeping values flowing while reusing scarce containers at the right time.

# Operations and Containers

We model the kernel as **operations** connected by **containers** that hold values:



- An operation **drains** input values from its input containers.
- It **fills** the result into one output container.

# When Can an Operation Start?

An operation is schedulable only when both conditions are true:

- ① **Input value-ready**: every input value it needs has been produced.
- ② **Output resource-free**: the container for its output is available.

In other words, the operation is blocked if either the data is not ready or the destination container is still occupied.

This is the basic rule behind SMEM ring-buffer reuse, TMEM reuse, and TMA store-buffer pipelining.

## Two Signals

In this model, synchronization has two meanings:

- ① **Data-ready**: a container is full; data is ready to be consumed.
- ② **Resource-free**: a container is empty; it can be reused.

A producer is waiting for a free output container. A consumer is waiting for a ready input value. Good GEMM scheduling is mostly about making those two waits overlap with useful work.

For TMEM, value readiness is gated by a completion condition: the **TMEM data ready** signal is sent only after the final MMA finishes.

—→ data/value flow

---→ synchronization signal

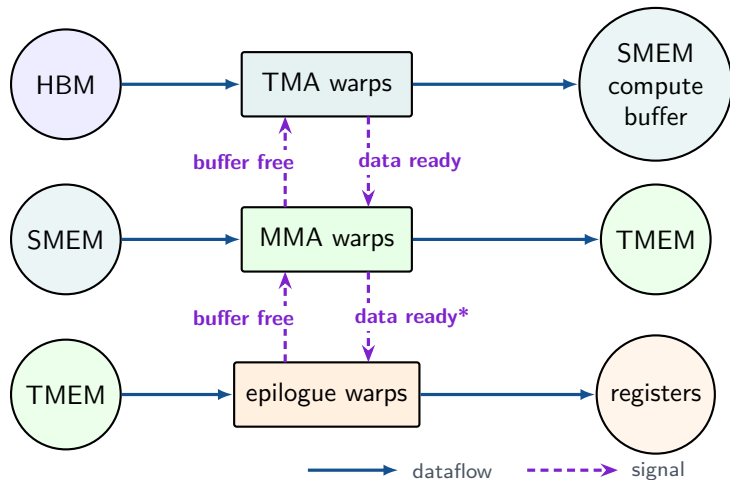
## Add the Containers

For the synchronization model used in this talk, we group the kernel into three producer/consumer operations:

| Operation      | Input container          | Output container         |
|----------------|--------------------------|--------------------------|
| TMA warps      | HBM A/B tiles            | SMEM compute ring buffer |
| MMA warps      | SMEM compute ring buffer | TMEM accumulator         |
| Epilogue warps | TMEM accumulator         | registers                |

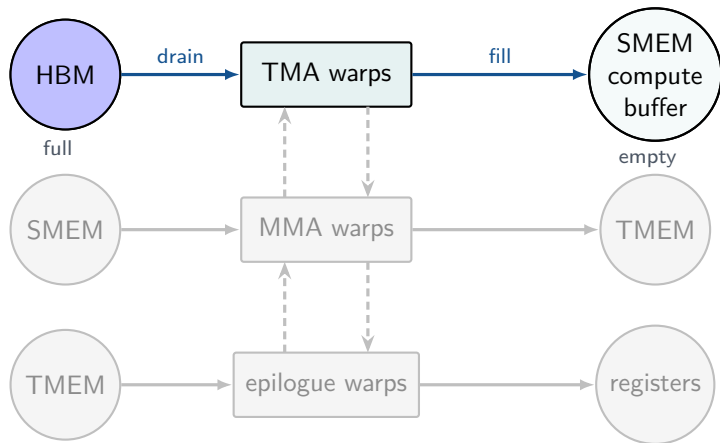
This is the abstraction used in the next graph. The epilogue's internal pack and TMA-store staging are real, but they are not the main warp-to-warp synchronization edges shown here.

# Main Synchronization Pattern: Graph View



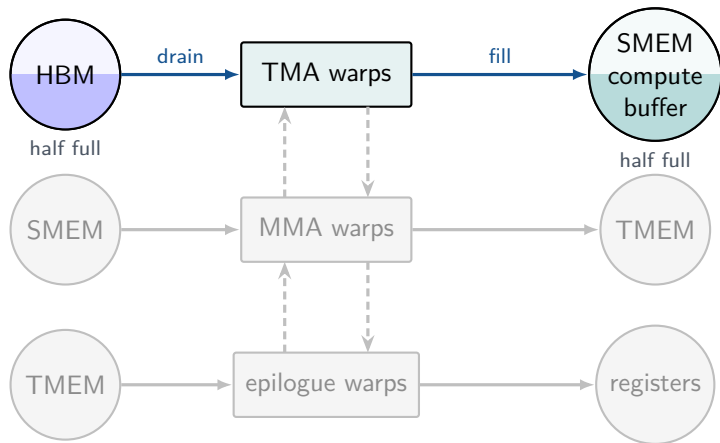
\* TMEM data ready is sent only after the final MMA finishes.

## TMA Warps: Fill SMEM, Signal Data Ready



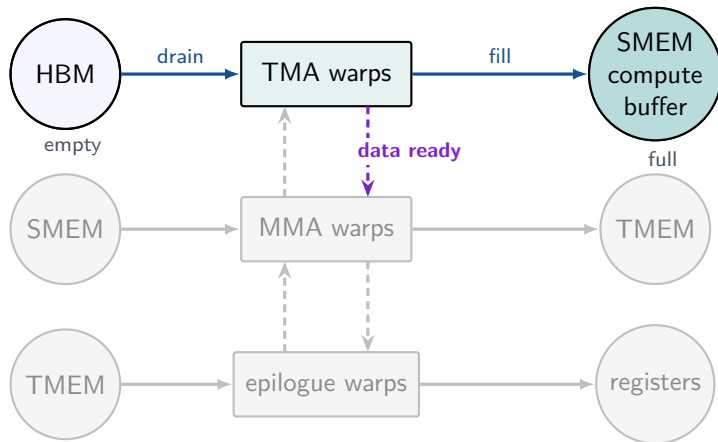
The TMA operation can start: the HBM tile is ready, and the SMEM compute slot is free.

## TMA Warps: Fill SMEM, Signal Data Ready



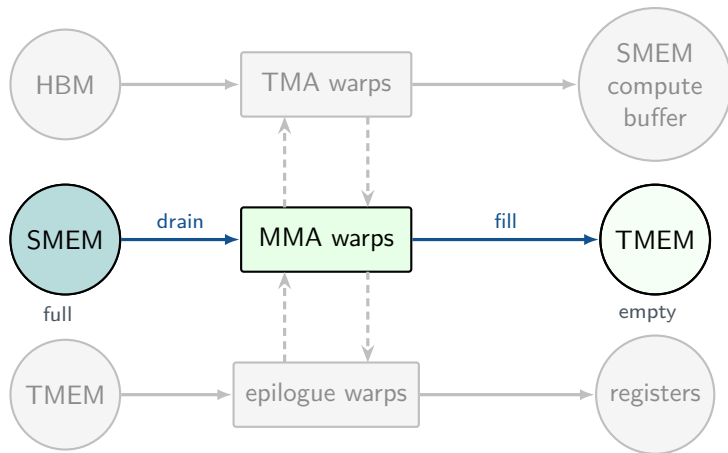
The TMA warps drain the HBM tile while filling the SMEM compute slot.

## TMA Warps: Fill SMEM, Signal Data Ready



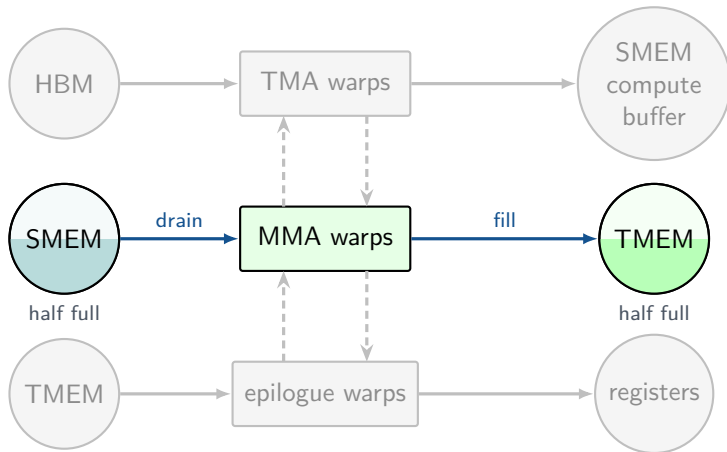
Once the HBM tile is drained and the SMEM slot is full, the TMA warps send **compute data ready** to the MMA warps.

## MMA Warps: Signal Buffer Free and Data Ready



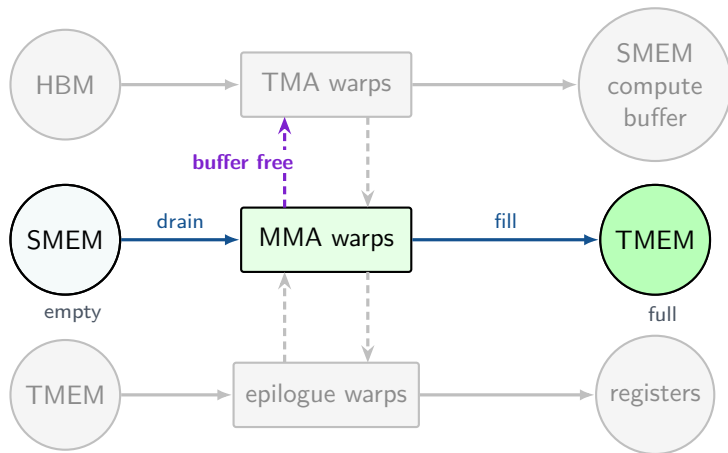
The MMA operation can start: compute data is ready in SMEM, and the TMEM accumulator buffer is free.

## MMA Warps: Signal Buffer Free and Data Ready



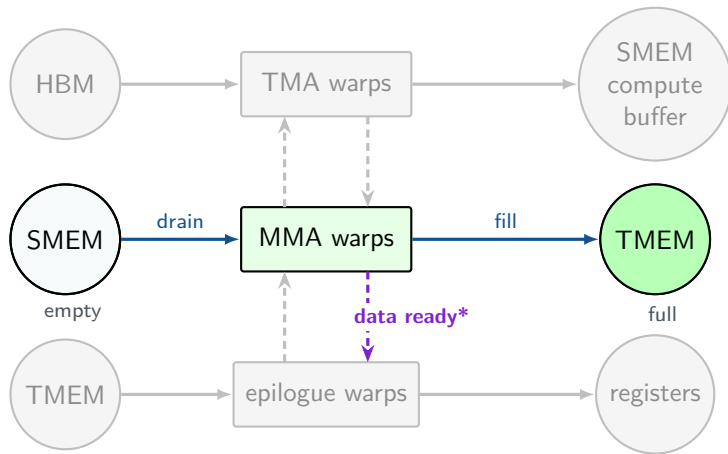
The MMA warps drain the SMEM compute buffer while filling TMEM.

## MMA Warps: Signal Buffer Free and Data Ready



Once the SMEM input is drained, the MMA warps send **compute buffer free** back to TMA.

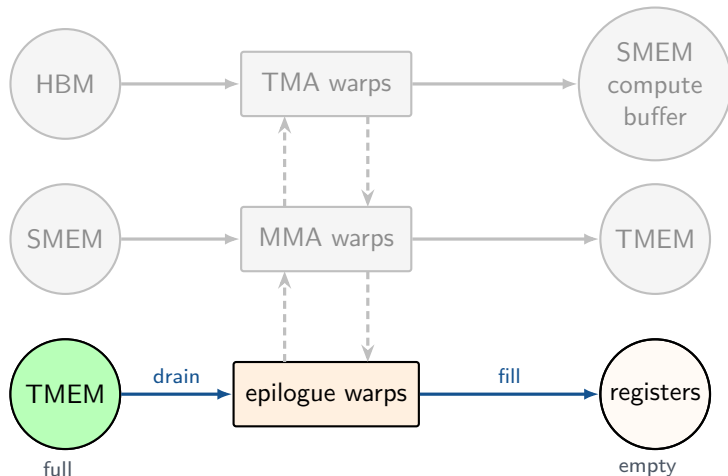
## MMA Warps: Signal Buffer Free and Data Ready



After the final MMA for this output tile, the MMA warps send **TMEM data ready** to the epilogue warps.

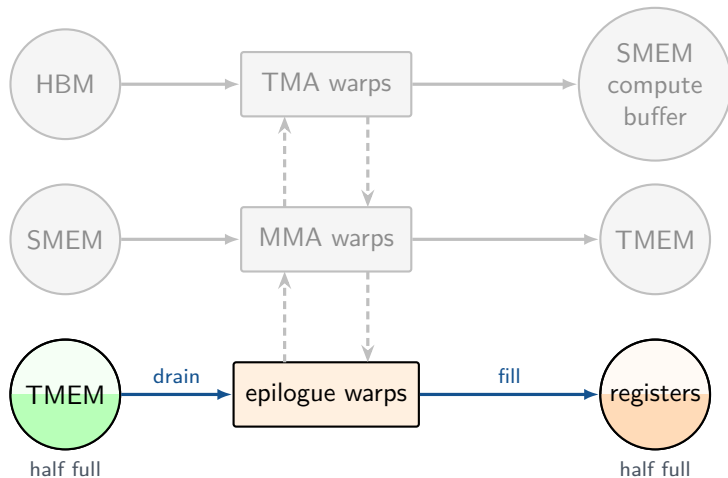
\* data ready is sent only after the final MMA finishes.

## Epilogue Warps: Drain TMEM, Signal Buffer Free



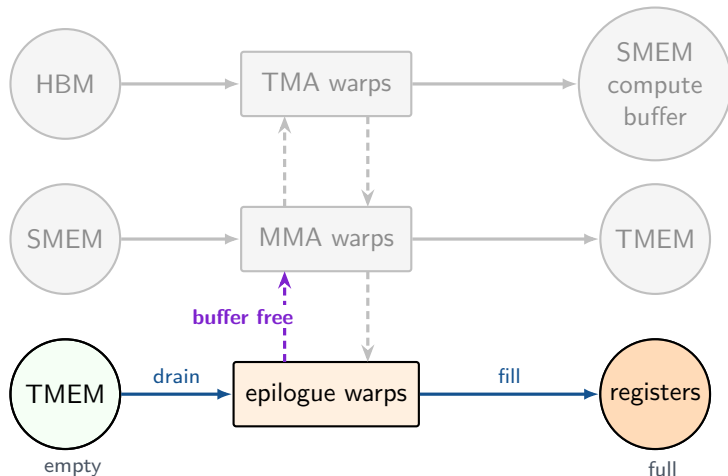
The epilogue can start: TMEM data is ready, and the register destination is available.

## Epilogue Warps: Drain TMEM, Signal Buffer Free



The epilogue warps drain TMEM while filling registers.

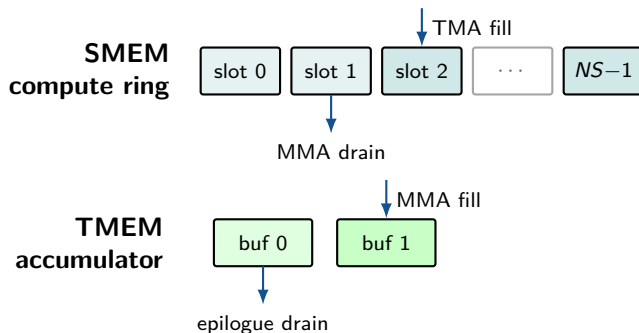
## Epilogue Warps: Drain TMEM, Signal Buffer Free



Once TMEM is drained, the epilogue warps send **TMEM buffer free** back to the MMA warps.

## Using Multiple Buffers

The single-container graph is the dependency for one slot. Real kernels replicate the same ready/free protocol across multiple slots.



| Container         | Buffering  | Effect                                                           |
|-------------------|------------|------------------------------------------------------------------|
| SMEM compute ring | $NS$ slots | MMA consumes one slot while TMA fills the next.                  |
| TMEM accumulator  | 2 buffers  | Epilogue drains the previous output while MMA produces the next. |

## From Diagram to Code

We now connect the container diagram to the real kernel. Each warp role maps to one loop that *waits*, *works*, then *signals*:

| Warp     | Waits on                     | Works             | Signals                           |
|----------|------------------------------|-------------------|-----------------------------------|
| TMA      | smem_compute_free_mbar[slot] | TMA load A/B      | smem_compute_full_mbar[slot]      |
| MMA      | smem_compute_full_mbar[slot] | tcgen05.mma       | smem_compute_free_mbar, tmem_full |
| Epilogue | tmem_full[buf]               | drain + TMA store | tmem_free_mbar[buf]               |

The mbarriers *are* the dashed signal arrows from the diagrams.

## TMA Warp: Fill SMEM, Signal Compute Data Ready

```
if (warp_id == 0 && elect_sync()) {           // TMA-load warp
    for (int k = 0; k < num_k; k++) {
        int slot = gk % NS;
        // wait: compute buffer free
        mbarrier_wait_phase(smem_compute_free_mbar[slot],
                             smem_compute_free_phase[slot]);
        tma_2d_load_g2(A_base(slot), A_tmap, ...); // drain HBM -> fill A
        tma_2d_load_g2(B_base(slot), B_tmap, ...); // fill B
        // signal: compute data ready
        mbarrier_arrive_expect_tx(smem_compute_full_mbar[slot],
                                   SLOT_BYTES);

        gk++;
    }
}
```

Wait for the output slot to be free, fill it, then signal that the compute data is ready to consume.

## MMA Warp: Compute Buffer Free, TMEM Data Ready

```
else if (cta_rank == 0 && warp_id == 1 && elect_sync()) {    // MMA warp
    mbarrier_wait_phase(tmem_free_mbar[buf],
                        tmem_free_phase[buf]);                // wait: TMEM buffer free
    for (int k = 0; k < num_k; k++) {
        int slot = gk % NS;
        // wait: compute data ready
        mbarrier_wait_phase(smem_compute_full_mbar[slot],
                            smem_compute_full_phase[slot]);
        issue_mma_chain(d_tmем, A_base(slot), B_base(slot), ...);
        // signal: compute buffer free
        tcgen05_commit_mcast_g2(smem_compute_free_mbar[slot],
                                cta_mask);

        gk++;
    }
    tcgen05_commit_mcast_g2(tmем_full[buf], cta_mask); // signal: TMEM data ready
}
```

`tmем_full` is the concrete mbarrier for *TMEM data ready*; it fires only after all MMAs for this tile are done.

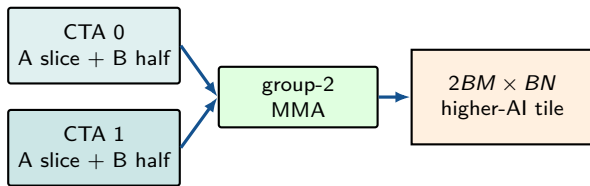
## Epilogue Warps: Drain TMEM, Signal TMEM Free

```
else if (warp_id >= 4 && warp_id < NUM_WARPS + 4) {    // epilogue warps
    mbarrier_wait_phase(tmem_full[buf], full[buf]);    // wait: TMEM data ready
    tcgen05_ld(...);                                  // drain TMEM -> registers
    // ... epilogue: write the result out ...
    EPI_TMEM_FREE_ARRIVE(buf);                          // signal: TMEM buffer free
}
```

At this point the epilogue is just the TMEM *consumer*; how it writes the result out is a design choice we cover later.

## One Key Optimization: 2-CTA MMA

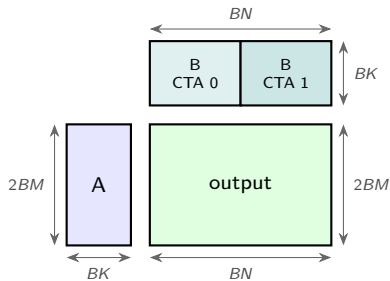
- Two CTAs form one cluster.
- Each CTA keeps one A slice and half of B in SMEM.
- One group-2 MMA produces a larger, higher-AI output tile.



Benefit: **higher arithmetic intensity**. The cluster computes a larger tile while B storage is split across the CTAs.

## 2-CTA MMA: Geometry

- Each CTA loads its  $BM \times BK$  slice of A.
- Each CTA loads a  $BK \times BN/2$  slice of B.
- At  $BN=256$ , per-CTA SMEM per slot drops from **48 KB** to **32 KB**.
- TMA loads use `.cta_group::2`; the `tcgen05.commit` multicasts the completion signal to both CTAs.



## 2-CTA MMA: In the Instructions

The cluster cooperation shows up directly in the PTX:

```
// TMA load into clustered SMEM (both CTAs' arrivals counted):
cp.async.bulk.tensor.2d.shared::cluster.global
    .mbarrier::complete_tx::bytes.cta_group::2    [smem], [tmap,{x,y}], [mbar];

// One MMA issued across the CTA pair:
tcgen05.mma.cta_group::2.kind::f16    ... ;

// Completion multicast to both CTAs in the cluster:
tcgen05.commit.cta_group::2.mbarrier::arrive::one
    .shared::cluster.multicast::cluster.b64    [smem_compute_free_mbar], cta_mask;
```

Source: [webui/kernels/tier3\\_cluster\\_swizzle/kernel.cu](https://github.com/NVIDIA/cuda-10.2/blob/master/source/runtime/ptx/kernels/tier3_cluster_swizzle/kernel.cu).

## 2-CTA MMA: More Compute Stages

Each CTA stores only **half** of B ( $BN_{\text{local}} = BN/2$ ), so the compute slot shrinks and **more pipeline stages fit** in the same SMEM.

**Single-CTA** (full B per CTA)

| $BN$ | B tile | slot  | stages |
|------|--------|-------|--------|
| 256  | 32 KB  | 48 KB | 4      |
| 512  | 64 KB  | 80 KB | 2      |

**2-CTA** (half B per CTA)

| $BN$ | B tile | slot  | stages   |
|------|--------|-------|----------|
| 256  | 16 KB  | 32 KB | <b>7</b> |
| 512  | 32 KB  | 48 KB | <b>4</b> |

A tile is 16 KB throughout. With the SMEM capacity ( $\approx 227$  KB),  $\text{stages} = \lfloor \text{SMEM} / \text{slot} \rfloor$ . 2-CTA fits  $\sim 2\times$  the stages.

# Tradeoff of Double TMEM Buffering

Tile width trades off **overlap** against **arithmetic intensity**.

$$\text{2-CTA MMA, } BM=128 : \quad AI = \frac{256BN}{256 + BN}$$

$BN=256$

## **MORE OVERLAP**

double TMEM buffer  
up to 7 compute stages  
lower AI: 128 flop/B

$BN=512$

## **HIGHER AI**

single TMEM buffer  
up to 4 compute stages  
delayed reuse: 171 flop/B

For medium-to-large square GEMMs, higher AI often wins.

## Epilogue Design: Three Phases

So far we have shown only the first phase of the epilogue. In full it has **three**:

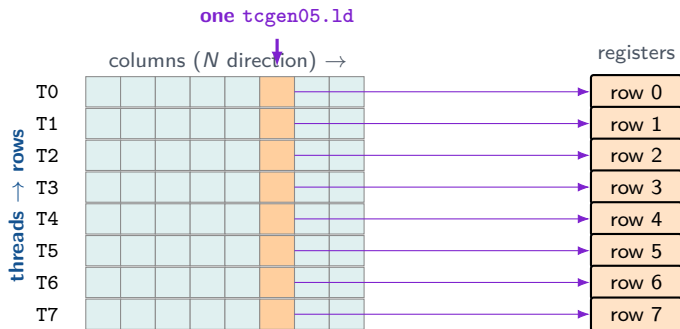


- ① **Drain TMEM into registers** with `tcgen05.ld` (already seen).
- ② **Write registers into an SMEM staging buffer.**
- ③ **Issue a TMA store** from SMEM to HBM.

Phases 2 and 3 are forced on us by *how TMEM is laid out*.

## Epilogue Design: TMEM Layout

TMEM is laid out for MMA accumulation: `tcgen05.1d` reads it **one column at a time**, with each **thread mapped to a row**.

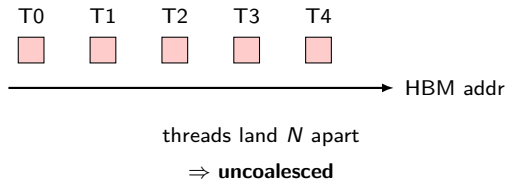


A load distributes one column across the warp's lanes; iterating, each **thread  $t$  ends up owning row  $t$**  — not a contiguous run of neighboring threads' outputs.

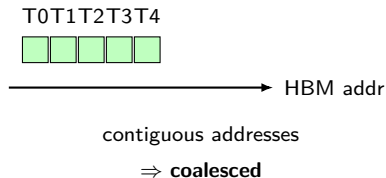
## Epilogue Concern 1: Coalesced Global Write

Each thread owns a *row*, so storing registers straight to HBM is row-strided. Instead we first fill one  $BM \times 64$  **chunk** in SMEM — the TMA store unit — then let **TMA** write it out, coalesced.

### Direct store



### Via SMEM staging



## Epilogue Concern 2: Filling the $128 \times 64$ SMEM Buffer

We fill it `tcgen05.1d.x8` chunk by chunk. One warp store writes **one int4 column** (one cell per lane — the highlighted column, not the whole row); naively those cells share **one bank**, so the swizzle  $n \oplus (\text{row} \& 7)$  spreads them.

**Naive: column  $\rightarrow$  fixed bank**

|    |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|
| T0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T2 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T3 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T4 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T5 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T6 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T7 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

a column hits **one bank**

$\Rightarrow$  8-way conflict

Cell =  $c \oplus r$  = bank group; each is one int4 = **4 banks** (8 cells = 32 banks) = the SWIZZLE\_128B atom.

Shown is one phase (lanes 0–7); A warp writes 32 rows and a 128-bit store issues as 4 identical phases.

**Swizzled:  $n \oplus (\text{row} \& 7)$**

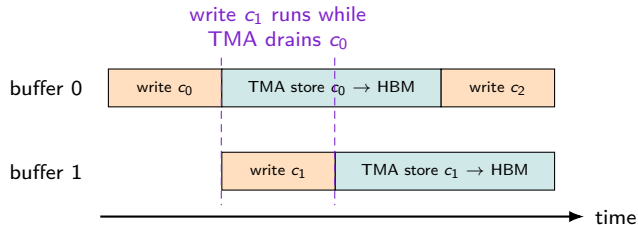
|    |   |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|---|
| T0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| T1 | 1 | 0 | 3 | 2 | 5 | 4 | 7 | 6 |
| T2 | 2 | 3 | 0 | 1 | 6 | 7 | 4 | 5 |
| T3 | 3 | 2 | 1 | 0 | 7 | 6 | 5 | 4 |
| T4 | 4 | 5 | 6 | 7 | 0 | 1 | 2 | 3 |
| T5 | 5 | 4 | 7 | 6 | 1 | 0 | 3 | 2 |
| T6 | 6 | 7 | 4 | 5 | 2 | 3 | 0 | 1 |
| T7 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |

each row  $\rightarrow$  a **different bank**

$\Rightarrow$  conflict-free

## Epilogue Design: Multiple Store Buffers

With  $\geq 2$  store buffers, one chunk's **TMA store** overlaps the next chunk's **SMEM write**:



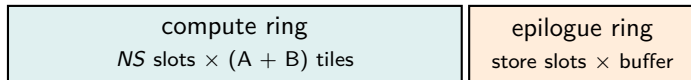
- The TMA store is **async**: once issued, the epilogue moves to the next chunk's write.
- With  $N$  buffers ( $N \in \{1, 2, 3, 4\}$ ) it fills a free one while TMA drains earlier ones — stalling only when all  $N$  are in flight.

## Epilogue Design: From Design to Code

The whole epilogue is one loop over the  $BN/64$  store chunks:

```
for (int chunk = 0; chunk < BN/64; chunk++) {  
    // Phase 1: load all 64 cols of this chunk into registers  
    for (int n = 0; n < 8; n++)           // 8 x tcgen05.ld.x8  
        tcgen05_ld_x8(tmem_row + chunk*64 + n*8, reg[n]);  
    tcgen05_wait_ld();  
    if (elect) tma_wait_group<TMA_STORE_STAGES-1>(); // a buffer is free  
    bar.sync;                                     // ... visible to all warps  
    // Phase 2: conflict-free SMEM write, then the TMA store  
    for (int n = 0; n < 8; n++) {  
        int swz = n ^ (row & 7);           // XOR swizzle: distinct banks  
        store_buf[stage][row][swz] = pack_bf16(reg[n]);  
    }  
    bar.sync;                                     // buffer fully written  
    if (elect) tma_store(store_buf[stage], C + chunk*64);  
    stage = (stage + 1) % NUM_STORE_BUFFERS;  
}
```

## Partitioning Shared Memory



- **Compute ring:**  $NS$  pipeline slots, each holding one A tile and one B tile for the MMA stage.
- **Epilogue ring:** store slots, each a TMA store buffer holding one  $BM \times 64$  bf16 chunk:  $128 \cdot 64 \cdot 2 = 16 \text{ KB}$ .
- Both rings share one per-SM SMEM budget, so larger tiles or deeper pipelines on one side leave less room for the other.

## Tile Sizes and the SMEM Budget

Per compute slot (2-CTA, each CTA owns half of B):

$$\text{slot} = \underbrace{BM \cdot BK \cdot 2}_{\text{A tile}} + \underbrace{\frac{BN}{2} \cdot BK \cdot 2}_{\text{B tile (local)}}$$

| Config (2-CTA, $BK=64$ ) | A tile | B tile | slot  |
|--------------------------|--------|--------|-------|
| $BM=128, BN=256$         | 16 KB  | 16 KB  | 32 KB |
| $BM=128, BN=512$         | 16 KB  | 32 KB  | 48 KB |

Budget constraint (per SM, B200  $\approx$  227 KB):

$$\underbrace{NS \times \text{slot}}_{\text{compute ring}} + \underbrace{E \times 16 \text{ KB}}_{\text{epilogue ring}} \leq S_{\text{SMEM}}$$

Example:  $BN=256, NS=5 \Rightarrow$  160 KB compute ring; with  $E=2$  store buffers, +32 KB = 192 KB total.

## Common Partitions for $BM=128$ (empirical)

Each split below is the **maximum** that fits in  $\approx 227$  KB; the config actually used is chosen by autotune per shape:

$BN=256$   up to  $6 \times 32 + 2 \times 16$  KB

$BN=256$   up to  $5 \times 32 + 4 \times 16$  KB

$BN=512$   up to  $4 \times 48 + 2 \times 16$  KB

 compute slot ( $BN=256$ : 32 KB,  $BN=512$ : 48 KB)  16 KB store buffer — fewer/larger compute slots trade for more store buffers.

# Autotuning: Configuration Search Space

Autotune sweeps these parameters and benchmarks every *valid* combo:

| Parameter (covered in this talk) | Candidate values |
|----------------------------------|------------------|
| $BM$ (tile rows)                 | {128}            |
| $BN$ (tile cols)                 | {256, 512}       |
| compute slots ( $NS$ )           | {2, 3, 4, 5, 6}  |
| epilogue slots (store buffers)   | {1, 2, 3, 4}     |

- A **static checker** prunes combos whose SMEM exceeds **227 KB** ( $NS \times \text{slot}(BN) + \text{store} \times 16 \leq 227$ ) before benchmarking; the fastest survivor wins.
- The framework fixes everything else; the one knob it leaves open is the **CTA-swizzle factor** {1, 2, 4, 8, 16} (an L2-reuse block ordering, not covered here).

## **Implementation and Performance Results**

**Warp layout — deliberately not a power of two.** Each CTA launches

$$\underbrace{1}_{\text{TMA}} + \underbrace{1}_{\text{MMA}} + \underbrace{4 \text{ or } 8}_{\text{epilogue}} = 6 \text{ or } 10 \text{ warps.}$$

- The **TMA** and **MMA** warps are fixed — one each.
- The **epilogue** warp count  $\in \{4, 8\}$  is **autotuned**; more warps give the TMEM drain and SMEM write more parallelism.
- So the CTA is sized to the work, not rounded up to a power of two.

## Performance: Square GEMMs vs cuBLAS

MMComposer averages **~100% of cuBLAS** (96–106% across 20 shapes). Each top config *is* an SMEM partition.

| shape              | cuBLAS | MMC  | %   | <i>BN</i> | <i>NS</i> | store |
|--------------------|--------|------|-----|-----------|-----------|-------|
| 4096 <sup>3</sup>  | 1382   | 1354 | 98  | 256       | 6         | 1     |
| 5120 <sup>3</sup>  | 1432   | 1373 | 96  | 512       | 4         | 2     |
| 6144 <sup>3</sup>  | 1475   | 1447 | 98  | 512       | 4         | 2     |
| 7168 <sup>3</sup>  | 1435   | 1397 | 97  | 256       | 5         | 1     |
| 8192 <sup>3</sup>  | 1420   | 1453 | 102 | 512       | 4         | 2     |
| 9216 <sup>3</sup>  | 1422   | 1426 | 100 | 512       | 4         | 2     |
| 10240 <sup>3</sup> | 1398   | 1390 | 99  | 512       | 3         | 1     |
| 11264 <sup>3</sup> | 1414   | 1440 | 102 | 512       | 3         | 2     |
| 12288 <sup>3</sup> | 1477   | 1472 | 100 | 512       | 4         | 2     |

TFLOPS (B200, BF16). *BN* / *NS* / **store** is the SMEM partition: *NS* compute slots of a *BN*-wide tile + *store* TMA buffers.

## Performance: Rectangular GEMMs vs cuBLAS

| shape ( $M \times N \times K$ )  | cuBLAS | MMC  | %   | $BN$ | $NS$ | store |
|----------------------------------|--------|------|-----|------|------|-------|
| $2048 \times 36864 \times 12288$ | 1517   | 1480 | 98  | 512  | 4    | 2     |
| $2048 \times 49152 \times 12288$ | 1366   | 1390 | 102 | 512  | 4    | 1     |
| $4096 \times 12288 \times 4096$  | 1486   | 1458 | 98  | 256  | 6    | 1     |
| $4096 \times 16384 \times 4096$  | 1512   | 1529 | 101 | 256  | 5    | 2     |
| $8192 \times 24576 \times 8192$  | 1490   | 1507 | 101 | 512  | 4    | 2     |
| $8192 \times 32768 \times 8192$  | 1404   | 1458 | 104 | 512  | 3    | 2     |
| $16384 \times 4608 \times 768$   | 1467   | 1432 | 98  | 256  | 5    | 1     |
| $32768 \times 4608 \times 768$   | 1461   | 1423 | 97  | 256  | 5    | 1     |
| $32768 \times 9216 \times 768$   | 1428   | 1415 | 99  | 256  | 6    | 1     |
| $32768 \times 12288 \times 768$  | 1321   | 1401 | 106 | 256  | 6    | 1     |
| $65536 \times 2304 \times 768$   | 1460   | 1424 | 98  | 256  | 5    | 1     |

Wide/short LLM-style shapes.  $BN$  /  $NS$  / **store** is the same SMEM partition as the square case — autotuned per shape.

# Why More Buffers Doesn't Always Help

The autotuned configs rarely max out the buffer counts (store mostly 1–2, not 4) — something we would not have predicted before seeing the numbers. Some intuition (*hypotheses*, not proven):

## Compute ring (more *NS*)

- Once the pipeline hides load/MMA latency, extra stages add nothing.

## Epilogue ring (more store buffers)

- If the epilogue is already fast relative to compute, the extra store $\leftrightarrow$ write overlap buys little.
- More TMA stores in flight = more memory parallelism that may contend with the TMA *loads* feeding the compute pipeline.

The optimum is shape-dependent and empirical — hence autotune.

## **Adding Epilogue Fusions**

## Adding Epilogue Fusions

A custom epilogue fusion is almost entirely an **epilogue** change — the compute pipeline is reused, with at most a minor TMA-load tweak.

**Case study: fused SwiGLU.** One GEMM produces two projections side by side in TMEM (*left* and *gate*); the epilogue writes out both and the gated result:

$$\text{SwiGLU} = \text{left} \odot \text{SiLU}(\text{gate}), \quad \text{SiLU}(g) = g \sigma(g).$$

- Loop over *pairs* of column chunks (a left chunk + its gate chunk).
- Per pair: load both into registers, then **three** stores — left, gate, and SwiGLU.
- *Dual-B*: left and gate are *separate* weight matrices; with 2-CTA on, each CTA loads its half from a different one — a tiny load change, with **no measured performance impact**.

## Fused SwiGLU: the Epilogue Loop

Only the epilogue changes; `stage_and_store` is the canonical swizzled-write + TMA-store helper:

```
for (int p = 0; p < (BN/64)/2; p++) {           // loop over chunk PAIRS
    for (int n = 0; n < 8; n++) {                 // load both projections
        tcgen05_ld_x8(tmem + p*64 + n*8, left[n]);
        tcgen05_ld_x8(tmem + (p+HALF)*64 + n*8, gate[n]);
    }
    tcgen05_wait_ld();

    stage_and_store(left, C, p*64);               // (1) store left
    stage_and_store(gate, C, (p+HALF)*64);        // (2) store gate

    for (int n = 0; n < 8; n++)                   // (3) fused SwiGLU math
        swig[n] = left[n] * silu_fast(gate[n]);  // ex2.approx + rcp.approx
    stage_and_store(swig, D, p*64);               // store SwiGLU -> D
}
```

`stage_and_store` = the same store step as the canonical epilogue; fusion just adds the math and an extra output tensor D.

## Fused SwiGLU: Almost Free

$M=32768$ ,  $N=4608$ ,  $K=768$ ;  $BM=128$ ,  $BN=256$ ,  $NS=6$ , store=2

|                                   | TFLOPS | time               |
|-----------------------------------|--------|--------------------|
| cuBLAS (GEMM only)                | 1406   | $165\ \mu\text{s}$ |
| fused SwiGLU ( $1.5\times$ store) | 1250   | $185\ \mu\text{s}$ |

- Despite  $1.5\times$  **the store traffic** (left + gate + SwiGLU) and the SwiGLU math, it is only  **$\sim 12\%$  slower** — 89% of a pure cuBLAS GEMM.
- The SwiGLU math is essentially **free**; the gap is almost entirely the **extra store**.

# Acknowledgements

Thanks to **Haidong** and **Gaoyuan** for the discussions and feedback that shaped this material.